JAVASCRIPT LAB PRACTICAL

30 Industry-Scenario Questions • Fundamentals, Operators, Control Statements & Loops

Coverage: Based on the uploaded JavaScript Fundamentals – Control Statements & Loops PPT (variables, data types, type conversion, operators, if/else, nested if, and for/while/do...while/for...of/for...in).

Important scope: Do not use switch, functions, DOM, events, or string/array methods. Use only the concepts covered in the PPT. A few questions use basic array/object literals because for...of and for...in are part of the lesson.

Instructions

• Write and run each solution in a separate JavaScript file or in the browser console.

• Use **const** by default and **let** when a value must change. Avoid **var**.

• Use strict comparisons (**===** and **!==**) unless a question specifically asks you to investigate loose equality.

• Validate boundary values such as 0, negative values, exact limits, empty strings, and values just above/below a limit.

• For loop questions, make sure the loop terminates. Do not intentionally run an infinite loop.

• For output-prediction questions, write the output first and then verify it by executing the code.

A. Business Logic & Real-World Scenarios (1–10)

1. E-commerce free-shipping eligibility

An online store offers free delivery when the cart total is at least ■999. Otherwise, delivery costs ■60. Read a cart total stored in a variable and print the payable amount and whether delivery is free.

Challenge / test cases: Test ■0, ■998, ■999 and ■1,500. Use if...else and arithmetic operators.

2. Subscription access checker

A SaaS platform grants access only when the subscription is active AND the account is not blocked. Store subscriptionStatus and isBlocked as variables and print “Access granted” or “Access denied”.

Challenge / test cases: Use Boolean values and && / !. Test all meaningful combinations.

3. Payment method validation

A checkout accepts payment if the customer has selected either UPI or a card. Store hasUPI and hasCard as Boolean values. Print a suitable result.

Challenge / test cases: Use ||. Then modify the rule: payment is allowed only if the account is verified AND at least one payment method is available.

4. Delivery fee by distance

A delivery application charges ■30 for distances up to 3 km, ■50 for distances above 3 km and up to 7 km, and ■80 above 7 km. Print the delivery fee for a given distance.

Challenge / test cases: Use an else-if ladder. Check exactly 3, 3.1, 7 and 7.1 km. Reject negative distance.

5. Employee attendance status

A company marks attendance as “Present” for 8 or more working hours, “Half Day” for at least 4 but less than 8 hours, and “Absent” for less than 4 hours.

Challenge / test cases: Use else-if. Treat negative hours and hours above 24 as invalid input.

6. Secure login decision (nested if)

Given a username, password, and accountLocked Boolean, decide whether login is successful. First check whether the username exists; then check whether the account is locked; finally validate the password.

Challenge / test cases: Use nested if statements. Print a distinct message for unknown user, locked account, wrong password and successful login.

7. Student result and scholarship eligibility

A student passes when marks are at least 40. A scholarship is awarded only when the student passes, scores at least 90, AND family income is below ■2,00,000.

Challenge / test cases: Use nested/combined conditions. Test marks 39, 40, 89, 90 and income exactly ■2,00,000.

8. Ride-booking surge eligibility

A ride app applies a 1.5x surge when demand is high AND available drivers are fewer than 5. Otherwise, the fare remains unchanged. Calculate and print the final fare.

Challenge / test cases: Use logical operators and arithmetic. Test demandHigh true/false and driver counts 4 and 5.

9. Cloud storage plan recommendation

A cloud service offers Basic for usage up to 5 GB, Standard for usage above 5 GB and up to 100 GB, and Premium above 100 GB. Display the recommended plan.

Challenge / test cases: Use else-if and validate that usage is not negative. Consider exact boundaries.

10. Online order risk flag

Flag an order for manual review if the order amount exceeds ■25,000 OR the billing country differs from the delivery country. Otherwise mark it as “Normal”.

Challenge / test cases: Use || and strict comparison. Test each condition independently and both together.

B. Loop-Based Practical Problems (11–20)

11. Generate invoice line numbers

An invoice contains a known number of items. Given itemCount, print line numbers from 1 through itemCount using a for loop.

Challenge / test cases: Test 0, 1 and 5. Ensure the final number is included exactly once.

12. Monthly savings projection

A user saves ■2,000 in the first month and adds ■500 more to the monthly saving amount every month. Print each month’s saving for 12 months and calculate the total saved.

Challenge / test cases: Use a for loop, an accumulator and arithmetic operators. Print the total after the loop.

13. Failed login attempt counter

A system allows a maximum of 3 failed attempts. Starting with failedAttempts = 0, simulate attempts using a while loop and stop when the limit is reached.

Challenge / test cases: Print the attempt number and a final lock message. Ensure the counter changes so the loop terminates.

14. Inventory stock countdown

A warehouse dispatches one unit at a time from a stock of 10 units. Use a while loop to print the remaining stock after each dispatch until stock reaches 0.

Challenge / test cases: Do not print negative stock. Explain why the update belongs inside the loop.

15. Minimum-one-run onboarding prompt

A console onboarding process must display “Welcome to setup” at least once and then check whether setup is complete. Demonstrate this using do...while with a condition that is initially false.

Challenge / test cases: Predict the number of executions before running the code. Explain why while would behave differently.

16. Generate a multiplication table

Given a productCode number, print its multiplication table from 1 to 10 in the format “7 x 1 = 7”.

Challenge / test cases: Use a for loop and string concatenation. Do not use functions or array methods.

17. Process a list of product prices

Given an array of product prices, use for...of to print each price and calculate the total inventory value.

Challenge / test cases: Use a running total. Do not use reduce(), forEach(), or other array methods.

18. Read configuration properties

Given an object containing appName, environment, version and debugMode, use for...in to print every property name and its corresponding value.

Challenge / test cases: Use object[key] to access values. Add one more property and confirm that the loop includes it.

19. Count eligible applicants

Given a fixed array of applicant ages, use for...of and an if condition to count how many applicants are 18 or older.

Challenge / test cases: Do not use filter() or length-based shortcuts beyond the loop's basic iteration. Print the final count.

20. Daily sales summary

Given an array of 7 daily sales amounts, use a loop to calculate total sales and count how many days achieved at least ■10,000.

Challenge / test cases: Print total sales, qualifying-day count and average sales. Use only basic arithmetic and loop concepts.

C. Logical Traps & Output Prediction (21–26)

21. Loose vs strict equality

Without running the code first, predict each output and explain the difference: console.log(5 == "5"); console.log(5 === "5"); console.log(5 != "5"); console.log(5 !== "5");

Challenge / test cases: Then explain why strict equality is preferred in application code.

22. Truthy/falsy checkout values

Predict Boolean() results for: 0, "0", "", "false", false, null, undefined, NaN and " ".

Challenge / test cases: Classify each as truthy or falsy. Pay special attention to non-empty strings.

23. Operator precedence trap

Predict the output of: console.log(10 + 2 * 3 > 15 && 8 / 2 === 4); and console.log(10 + 2 * (3 > 1));

Challenge / test cases: If the second expression behaves unexpectedly, identify the type involved and explain the evaluation order.

24. Short-circuit logic reasoning

Given let isMember = false; let cartTotal = 2000; predict the result of isMember && cartTotal > 1000 and isMember || cartTotal > 1000.

Challenge / test cases: Change the values and create one case where both expressions are false and one where both are true.

25. Boundary-condition bug hunt

A developer writes: if (score > 40) { console.log("Pass"); } else { console.log("Fail"); }. The business rule says 40 is also a pass mark. Identify and correct the bug.

Challenge / test cases: Test 39, 40 and 41. Explain why boundary testing matters.

26. Loop tracing challenge

Predict the exact output and final values before execution: let a = 1; for (let i = 0; i < 4; i++) { console.log(a++); } console.log(a);

Challenge / test cases: Explain the difference between post-increment and the value printed by the loop.

D. Interview-Level & Debugging Challenges (27–30)

27. for...of versus for...in

Given const roles = ["Admin", "Editor", "Viewer"], write one loop using for...of to print role values and another using for...in to print indexes. Explain why using for...in when you need array values is a poor choice.

Challenge / test cases: Also state what for...in is intended to enumerate in the lesson's object example.

28. Off-by-one and direction error

A developer wants to print 1 through 5 but writes for (let i = 1; i < 5; i++) { console.log(i); }. Correct it. Then explain what happens if i++ is changed to i--.

Challenge / test cases: Identify the missing boundary value and explain the risk of a non-terminating loop.

29. Choose the correct loop

For each requirement, select and implement the most suitable loop: (a) print 1–20, (b) repeat until a balance becomes zero, (c) display a prompt at least once, (d) visit every value in an array, (e) inspect every key in an object.

Challenge / test cases: Give one sentence justifying each loop choice: for, while, do...while, for...of or for...in.

30. Mini project – checkout eligibility engine

Build a console-based checkout decision using variables for cartTotal, stockAvailable, customerVerified, accountBlocked and deliveryDistance. Reject the order if stock is unavailable, the customer is not verified, or the account is blocked. If valid, calculate delivery fee using the distance slabs from Question 4 and apply free delivery for cart totals of at least ■999. Print a clear order summary.

Challenge / test cases: Use variables, type conversion if needed, arithmetic, strict comparisons, logical operators, if/else-if, nested if and at least one loop to process a small set of order quantities or product prices. Do not use switch, functions, DOM or events.

Suggested evaluation rubric (optional)

Criteria	Marks / focus
Correct logic and conditions	40%
Correct loop selection and termination	20%
Boundary cases and logical accuracy	20%
Readable code, naming and formatting	10%
Explanation and output verification	10%

Trainer note: Questions 1–20 emphasize implementation; 21–26 are deliberately designed to expose type-coercion, truthiness, precedence and boundary misunderstandings; 27–30 focus on interview reasoning, debugging and combining concepts.